

Classwork – ML1

Session 1: From Data to Feature, Label, Scaling and Evaluation

موضوع کلی

هدف این است که با مفاهیم زیر بهتر آشنا شوید:

- Dataset
- Sample
- Feature
- Label
- Rule-based AI
- Machine Learning
- Deep Learning
- Classification
- Regression
- Feature Scaling
- Distance
- Accuracy
- Absolute Error

راهنمای سطح سوال‌ها

Core: ● سوال‌های اصلی که همه باید انجام دهند

Stretch: ● سوال‌های کمی چالشی‌تر

Bonus: 🎁 سوال اختیاری برای گروه‌هایی که زمان اضافه دارند

● بخش 1 Warm-up: Dataset, Sample, Feature, Label

داده‌ی زیر اطلاعات ۸ دانشجو را نشان می‌دهد.

ستون‌ها به ترتیب عبارت‌اند از:

```
# study_hours, attendance_percent, previous_score, final_score

students = np.array([

    [2, 60, 45, 50],

    [5, 80, 70, 75],

    [1, 40, 35, 38],

    [8, 90, 88, 92],

    [4, 75, 60, 65],

    [7, 85, 80, 84],

    [3, 55, 50, 52],

    [6, 70, 72, 78]

])
```

به سوال‌های زیر پاسخ دهید:

1. مقدارهای `ndim`, `shape`, `size`, `dtype` را برای آرایه‌ی `students` چاپ کنید.

2. چند `sample` در این `dataset` وجود دارد؟

3. چند ستون یا `feature` در این `dataset` وجود دارد؟

4. سه ستون اول را جدا کنید و در متغیری به نام `X` ذخیره کنید.

5. ستون آخر یعنی `final_score` را جدا کنید و در متغیری به نام `y` ذخیره کنید.

6. مقدارهای `X.shape` و `y.shape` را چاپ کنید.

7. در یک `comment` کوتاه بنویسید:

- هر row در این dataset نشان‌دهنده‌ی چیست؟

- هر column نشان‌دهنده‌ی چیست؟

- در این مثال، `X` چیست و `y` چیست؟

● بخش 2 Rule-based AI – یا Machine Learning ؟

در این بخش می‌خواهیم تفاوت بین Rule-based AI و Machine Learning را بهتر بفهمیم.

فرض کنید می‌خواهیم براساس `final_score` مشخص کنیم دانشجو قبول شده یا مردود.

اگر نمره‌ی نهایی دانشجو ۶۰ یا بیشتر باشد، دانشجو `pass` است.

در غیر این صورت، دانشجو `fail` است.

1. با استفاده از `np.where` یک آرایه به نام `rule_based_label` بسازید.

```
# if final_score >= 60 => pass
```

```
# else => fail
```

2. آرایه‌ی `rule_based_label` را چاپ کنید.

3. تعداد دانشجویان `pass` و `fail` را حساب کنید.

4. در یک `markdown cell` توضیح دهید:

- چرا این روش را می‌توان یک روش Rule-based در نظر گرفت؟

- چرا این روش هنوز یک مدل واقعی Machine Learning نیست؟

5. حالا فرض کنید به‌جای اینکه خودمان قانون بنویسیم، می‌خواهیم یک مدل از روی

`feature` های زیر الگو را یاد بگیرد:

study_hours

attendance_percent

previous_score

در یک جمله توضیح دهید در این حالت نقش Machine Learning چیست.

● بخش 3 Feature Engineering – ساده

در این بخش می‌خواهیم یک feature جدید از روی feature های قبلی بسازیم.

از ماتریس X استفاده کنید.

ستون‌ها به ترتیب عبارت‌اند از:

study_hours, attendance_percent, previous_score

1. میانگین study_hours را حساب کنید.

2. بیشترین و کمترین مقدار attendance_percent را حساب کنید.

3. یک feature جدید به نام effort_score بسازید:

$$\text{effort_score} = \text{study_hours} * \text{attendance_percent} / 100$$

4. مقدار effort_score را برای همه‌ی دانشجویها چاپ کنید.

5. این feature جدید را به X اضافه کنید و ماتریس جدیدی به نام X_{new} بسازید.

راهنمایی:

$\text{effort_score.reshape}(-1, 1)$

$\text{np.concatenate}(\dots)$

یا

$\text{np.column_stack}(\dots)$

6. مقدار $X_{\text{new}}.\text{shape}$ را چاپ کنید.

7. در یک comment کوتاه توضیح دهید چرا ساختن feature جدید می‌تواند در Machine Learning مفید باشد.

● بخش 4 Regression – یا Classification ؟

در حالت فعلی، مقدار `final_score` یک عدد است.

پس اگر بخواهیم مقدار دقیق `final_score` را پیش‌بینی کنیم، مسئله از نوع Regression است.

حالا می‌خواهیم از روی `final_score` یک Label دسته‌ای بسازیم.

قانون:

- اگر `final_score >= 60` باشد، `label` برابر "pass" است.
- در غیر این صورت، `label` برابر "fail" است.
- 1. با استفاده از `np.where` آرایه‌ای به نام `class_label` بسازید.
- 2. مقدار `class_label` را چاپ کنید.
- 3. تعداد دانشجویهای `pass` و `fail` را حساب کنید.
- 4. درصد قبولی کلاس را حساب کنید.
- 5. در یک markdown cell توضیح دهید:

- چرا پیش‌بینی `final_score` یک مسئله‌ی Regression است؟
 - چرا پیش‌بینی `pass/fail` یک مسئله‌ی Classification است؟
-

● بخش 5 Multi-class Label –

حالا می‌خواهیم به جای دو `label`، سه `label` داشته باشیم.

قانون:

- اگر `final_score < 60` باشد، `label` برابر "weak" است.

- اگر $80 < \text{final_score} \leq 60$ باشد، label برابر "normal" است.
 - اگر $\text{final_score} \geq 80$ باشد، label برابر "strong" است.
1. با استفاده از `np.where` یا Boolean Masking یک آرایه به نام `level_label` بسازید.
 2. آرایه‌ی `level_label` را چاپ کنید.
 3. تعداد دانشجویهای هر کلاس را حساب کنید.
 4. میانگین `study_hours` را برای هر کلاس جداگانه حساب کنید.
 5. خروجی را به شکل زیر چاپ کنید:

weak students average study hours: ...

normal students average study hours: ...

strong students average study hours: ...

6. در یک comment بنویسید این مسئله Binary Classification است یا Multi-class Classification.

● بخش 6 Feature Scaling –

در Machine Learning گاهی feature ها scale های متفاوتی دارند.

مثلا در dataset ما:

- `study_hours` بین ۱ تا ۸ است.
 - `attendance_percent` بین ۴۰ تا ۹۰ است.
 - `previous_score` بین ۳۵ تا ۸۸ است.
- اگر بخواهیم distance حساب کنیم، feature هایی که عددهای بزرگتری دارند ممکن است تاثیر بیشتری روی distance بگذارند.

1. برای هر ستون از X مقدارهای زیر را حساب کنید:

min

max

mean

std

2. با استفاده از Min-Max Scaling آرایه‌ای به نام `X_scaled` بسازید.

فرمول:

$$X_scaled = (X - X_min) / (X_max - X_min)$$

3. مقدار `X_scaled` را چاپ کنید.

4. بررسی کنید که همه‌ی مقدارهای `X_scaled` بین ۰ و ۱ هستند.

راهنمایی:

`np.min(X_scaled)`

`np.max(X_scaled)`

5. در یک `comment` توضیح دهید چرا قبل از محاسبه‌ی `distance` بهتر است `feature` ها را `scale` کنیم.

● بخش 7 Distance – و Nearest Neighbor

فرض کنید یک دانشجوی جدید داریم:

```
new_student = np.array([5, 75, 68])
```

این سه عدد به ترتیب نشان‌دهنده‌ی موارد زیر هستند:

```
# study_hours, attendance_percent, previous_score
```

می‌خواهیم ببینیم این دانشجو به کدام دانشجوی قبلی شبیه‌تر است.

1. دانشجوی جدید را با همان `X_min` و `X_max` مربوط به بخش قبل `scale` کنید.

2. بدون استفاده از loop ، فاصله ی Euclidean بین new_student_scaled و همه ی دانشجویهای قبلی را حساب کنید.

فرمول: Euclidean Distance

```
distance = sqrt(sum((a - b) ** 2))
```

راهنمایی در: NumPy

```
np.linalg.norm(...)
```

3. آرایه ی distance ها را چاپ کنید.

4. با استفاده از np.argmin نزدیک ترین دانشجو را پیدا کنید.

5. اطلاعات دانشجوی نزدیک را چاپ کنید.

6. final_score دانشجوی نزدیک را چاپ کنید.

7. class_label دانشجوی نزدیک را چاپ کنید.

8. در یک markdown cell توضیح دهید این روش شبیه ایده ی کدام الگوریتم Machine Learning است.

● بخش 8 Simple Prediction with Nearest Neighbor –

در این بخش می خواهیم یک prediction ساده انجام دهیم.

ایده:

برای هر دانشجوی جدید، نزدیک ترین دانشجوی قبلی را پیدا می کنیم و label همان دانشجو را به او نسبت می دهیم.

داده های جدید:


```
new_students = np.array([
    [2, 50, 40],
    [6, 82, 78],
    [4, 65, 58],
    [8, 88, 85]
])
```

1. همه‌ی `new_students` را با همان `X_min` و `X_max` مربوط به `dataset` اصلی `scale` کنید.

2. فاصله‌ی هر دانشجوی جدید تا همه‌ی دانشجویان قبلی را بدون `loop` حساب کنید.

راهنمایی:

خروجی `distance` باید `shape` زیر را داشته باشد:

(4, 8)

یعنی:

- ۴ دانشجوی جدید

- ۸ دانشجوی قبلی

3. برای هر دانشجوی جدید، نزدیک‌ترین `sample` را پیدا کنید.

4. `label` پیش‌بینی‌شده برای هر دانشجوی جدید را چاپ کنید.

5. خروجی باید شبیه این باشد:

Student 1 predicted label: ...

Student 2 predicted label: ...

Student 3 predicted label: ...

Student 4 predicted label: ...

● بخش 9 Evaluation with Accuracy –

فرض کنید label واقعی دانشجویان جدید این است:

```
true_labels = np.array(["fail", "pass", "pass", "pass"])
```

و prediction های شما از بخش قبل در متغیر زیر ذخیره شده است:

```
predicted_labels
```

1. مقدارهای predicted_labels و true_labels را چاپ کنید.

2. با مقایسه‌ی این دو آرایه، یک آرایه‌ی Boolean بسازید که نشان دهد کدام prediction درست بوده است.

3. تعداد prediction های درست را حساب کنید.

4. مقدار Accuracy را حساب کنید.

فرمول:

```
accuracy = number_of_correct_predictions / total_number_of_predictions
```

5. مقدار accuracy را به درصد چاپ کنید.

6. در یک markdown cell توضیح دهید:

○ Accuracy چه چیزی را اندازه‌گیری می‌کند؟

○ اگر accuracy برابر 0.75 باشد یعنی چه؟

● بخش 10 Regression Error – Absolute Error

در این بخش مسئله از نوع Regression است، چون می‌خواهیم عدد واقعی final_score را پیش‌بینی کنیم.

فرض کنید مقدار واقعی و مقدار پیش‌بینی‌شده‌ی مدل به شکل زیر است:

```
actual_scores = np.array([50, 75, 38, 92, 65])
```

```
predicted_scores = np.array([55, 70, 45, 88, 60])
```

1. خطای هر sample را حساب کنید:

```
error = predicted_scores - actual_scores
```

2. مقدار Absolute Error را برای هر sample حساب کنید.

فرمول Absolute Error برای هر sample:

```
absolute_error_i = abs(predicted_i - actual_i)
```

در: NumPy

```
absolute_errors = np.abs(predicted_scores - actual_scores)
```

3. مقدار absolute_errors را چاپ کنید.

4. مقدار Mean Absolute Error یا MAE را حساب کنید.

فرمول:

```
MAE = mean(abs(predicted - actual))
```

5. بیشترین error مربوط به کدام sample است؟

6. در یک markdown cell توضیح دهید:

- چرا این بخش مربوط به Regression است؟
- چرا فقط نگاه کردن به error معمولی کافی نیست؟
- چرا از Absolute Error استفاده می‌کنیم؟

● بخش 11 Supervised, Unsupervised, Semi-supervised, Reinforcement

در این بخش چند dataset کوچک داریم. برای هر کدام مشخص کنید مربوط به کدام نوع learning است.

گزینه‌ها:

Supervised Learning

Unsupervised Learning

Semi-supervised Learning

Reinforcement Learning

حالت اول

```
# features: study_hours, attendance
```

```
X1 = np.array([
```

```
    [2, 60],
```

```
    [5, 80],
```

```
    [1, 40],
```

```
    [8, 90]
```

```
])
```

```
# label: pass/fail
```

```
y1 = np.array(["fail", "pass", "fail", "pass"])
```

حالت دوم

```
# customer features without labels
```

```
X2 = np.array([
```

```
    [22, 2, 15],
```

```
    [25, 4, 20],
```

```
    [52, 12, 150],
```

```
[55, 11, 160]
```

```
)
```

حالت سوم

```
# some samples have labels, some do not
```

```
X3 = np.array([
```

```
    [2, 60],
```

```
    [5, 80],
```

```
    [1, 40],
```

```
    [8, 90]
```

```
])
```

```
y3 = np.array(["fail", "pass", "unknown", "unknown"])
```

حالت چهارم

```
# agent takes actions and receives rewards
```

```
actions = np.array(["left", "right", "jump", "right"])
```

```
rewards = np.array([-1, 2, 5, 3])
```

سوال‌ها:

1. برای هر حالت مشخص کنید مربوط به کدام نوع learning است.

2. برای هر حالت در یک جمله دلیل خود را بنویسید.

3. کدام حالت label کامل دارد؟

4. کدام حالت label ندارد؟

5. کدام حالت با reward سروکار دارد؟

● بخش 12 – AI, ML, DL Comparison with Code

در این بخش می‌خواهیم تفاوت کلی AI, ML, و DL را به صورت مفهومی مرور کنیم.

فرض کنید سه روش برای تشخیص pass/fail داریم.

روش اول Rule-based AI :

```
# If final_score >= 60, predict pass
```

روش دوم Machine Learning :

```
# Learn pattern from features:
```

```
# study_hours, attendance_percent, previous_score
```

روش سوم Deep Learning :

```
# Use many layers and many parameters to learn complex patterns
```

سوال‌ها:

1. روش اول را با np.where پیاده‌سازی کنید.
2. در یک markdown cell توضیح دهید چرا روش اول Rule-based است.
3. در یک markdown cell توضیح دهید در ML قانون را چه کسی می‌نویسد: انسان یا مدل؟
4. در یک markdown cell توضیح دهید DL چه زمانی مفیدتر است.
5. با توجه به dataset کوچک این تمرین، آیا واقعا به Deep Learning نیاز داریم؟ چرا؟

● بخش 13 – جمع‌بندی Session 1

در یک markdown cell به سوال‌های زیر پاسخ دهید:

1. Dataset چیست؟

2. Sample چیست؟

3. Feature چیست؟
 4. Label چیست؟
 5. تفاوت Classification و Regression چیست؟
 6. چرا Feature Scaling مهم است؟
 7. چرا Distance در بعضی روش‌های ML مهم است؟
 8. Accuracy برای چه نوع مسئله‌ای استفاده شد؟
 9. Absolute Error برای چه نوع مسئله‌ای استفاده شد؟
 10. تفاوت Machine Learning و Rule-based AI چیست؟
 11. تفاوت کلی Machine Learning و Deep Learning چیست؟
 12. هدف اصلی مدل ML چیست: حفظ کردن داده‌ها یا یاد گرفتن pattern برای prediction بهتر؟
-

Bonus – Function Design

اگر زمان اضافه داشتید، تابع زیر را کامل کنید.

```
def min_max_scale(data):
```

```
    """
```

```
    input:
```

```
        data: a NumPy array
```

```
    output:
```

```
        scaled_data
```

```
        data_min
```

```
data_max
```

```
"""
```

سپس تابع را روی X تست کنید و خروجی را چاپ کنید.

بعد از آن، تابع دیگری بنویسید:

```
def calculate_accuracy(y_true, y_pred):
```

```
"""
```

```
input:
```

```
    y_true: true labels
```

```
    y_pred: predicted labels
```

```
output:
```

```
    accuracy
```

```
"""
```

```
# write your code here
```

تابع را روی داده‌های بخش Accuracy تست کنید.